D20A **Roll No.19**

Experiment No. 9

Aim:

To create an ERC 20 token using Solidity in Remix IDE and deploy the token contract to a selected testnet using a MetaMask wallet. The experiment also studies basic ERC 20 functions, events, and attributes.

Theory:

An ERC-20 token is a fungible digital asset that follows a standard blockchain interface for managing token balances, transfers, and spending permissions. The `MyToken` contract specifies public properties such as token name, symbol, decimal precision, and total supply, along with mappings for account balances and approved allowances. It generates `Transfer` and `Approval` events whenever tokens are transferred or spending permissions are granted.

The contract implements three major functions: `transfer` for sending tokens between accounts, `approve` for authorizing another account to spend tokens, and `transferFrom` for executing transfers using an approved allowance. During deployment, the constructor initializes the token supply and assigns it to the deployer's account. Remix IDE is used for compiling and deploying the contract, while MetaMask handles transaction authorization on the selected test network.

ERC-20 is designed for fungible tokens, whereas ERC-721 is used for non-fungible tokens (NFTs), where each token represents a uniquely identifiable asset.

Steps:

1. Open the LearnETH Solidity tutorials in Remix IDE and explore the ERC 20 Token course.
2. Study the six chapters provided in the ERC 20 Token course.
3. Set up a MetaMask wallet and select a supported testnet.
4. Fund the wallet with testnet cryptocurrency using an available faucet.
5. Create the Solidity file MyToken.sol and enter the ERC 20 token contract.
6. Compile MyToken.sol using a compatible Solidity compiler in Remix IDE.
7. Select the browser wallet deployment environment and connect MetaMask to the selected testnet.
8. Enter the initial token supply and approve the deployment transaction in MetaMask.
9. Deploy MyToken and inspect the deployed contract and transaction information.
10. Use the public token functions and attributes to inspect the token name, symbol, decimals, total supply, and balances.

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract MyToken {
    string public name = "Dipesh Token";
    string public symbol = "Dipesh";
    uint8 public decimals = 18;
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;
    event Transfer(
        address indexed from,
```

Experiment No. 9

```
    address indexed to,
    uint256 value
);
event Approval(
    address indexed owner,
    address indexed spender,
    uint256 value
);
constructor(uint256 initialSupply) {
    totalSupply = initialSupply * 10 ** decimals;
    balanceOf[msg.sender] = totalSupply;
    emit Transfer(address(0), msg.sender, totalSupply);
}
function transfer(
    address to,
    uint256 amount
) public returns (bool) {
    require(
        balanceOf[msg.sender] >= amount,
        "Insufficient balance"
    );
    balanceOf[msg.sender] -= amount;
    balanceOf[to] += amount;
    emit Transfer(msg.sender, to, amount);
    return true;
}
function approve(
    address spender,
    uint256 amount
) public returns (bool) {
    allowance[msg.sender][spender] = amount;
    emit Approval(msg.sender, spender, amount);
    return true;
}
function transferFrom(
    address from,
    address to,
    uint256 amount
) public returns (bool) {
    require(
        balanceOf[from] >= amount,
        "Insufficient balance"
    );
    require(
        allowance[from][msg.sender] >= amount,
        "Allowance exceeded"
    );
    balanceOf[from] -= amount;
    balanceOf[to] += amount;
    allowance[from][msg.sender] -= amount;
```

Experiment No. 9

```
        emit Transfer(from, to, amount);
        return true;
    }
}
```

Listing 1. MyToken.sol

Output:

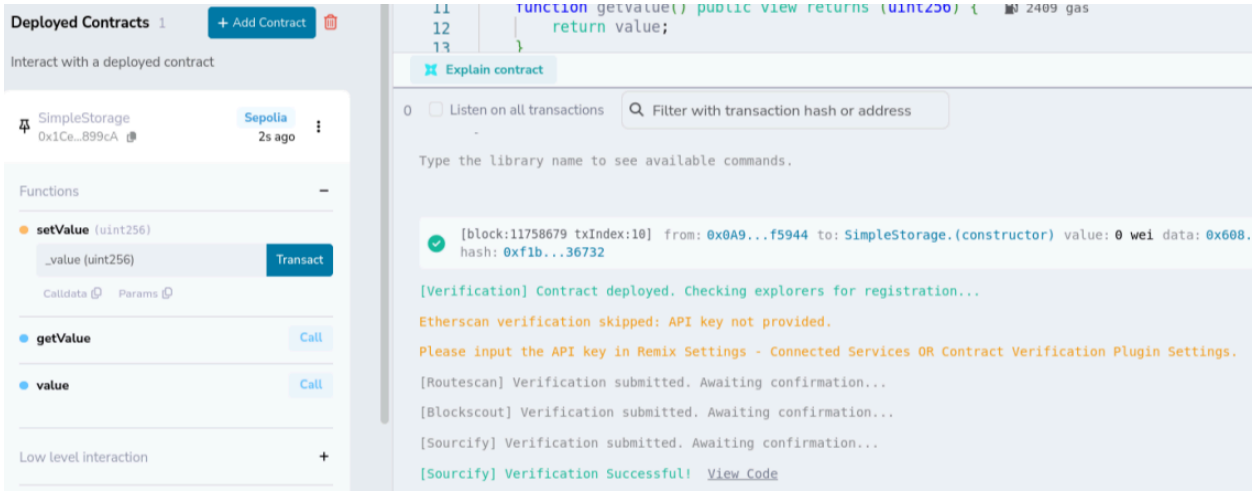


Fig. 1. MetaMask transaction request for deploying MyToken

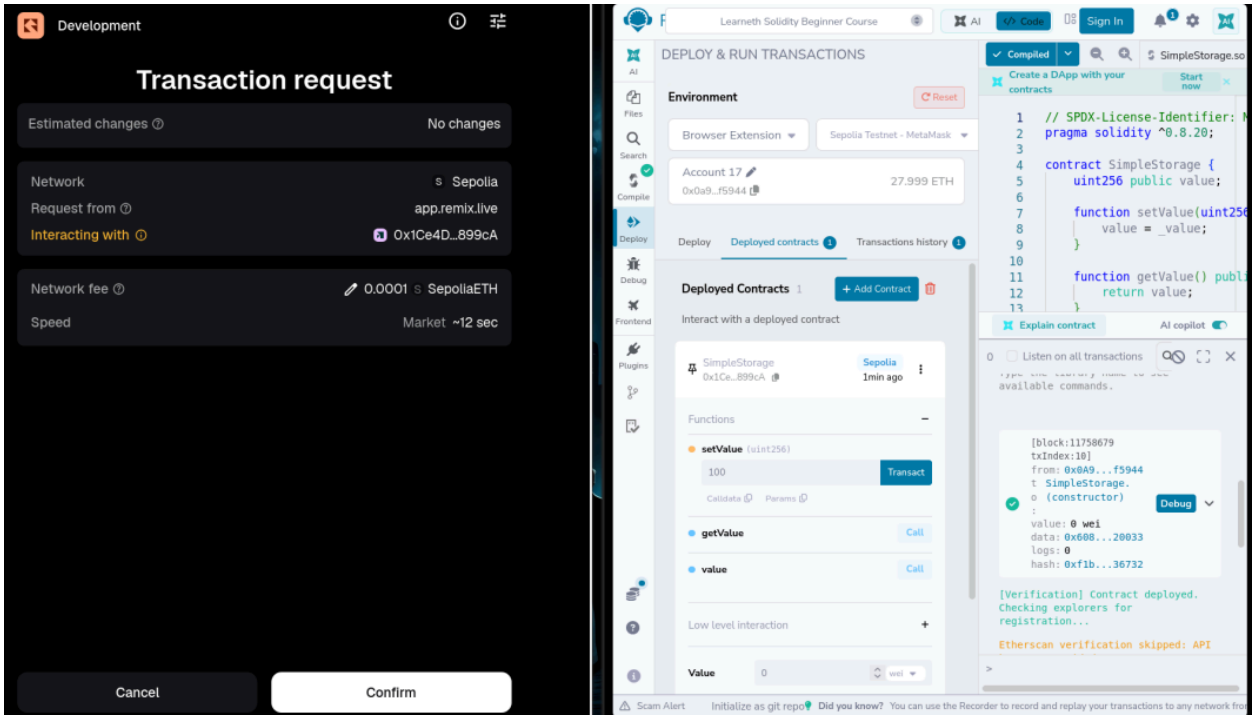


Fig. 2. MyToken deployment request and Remix deployment panel

Experiment No. 9

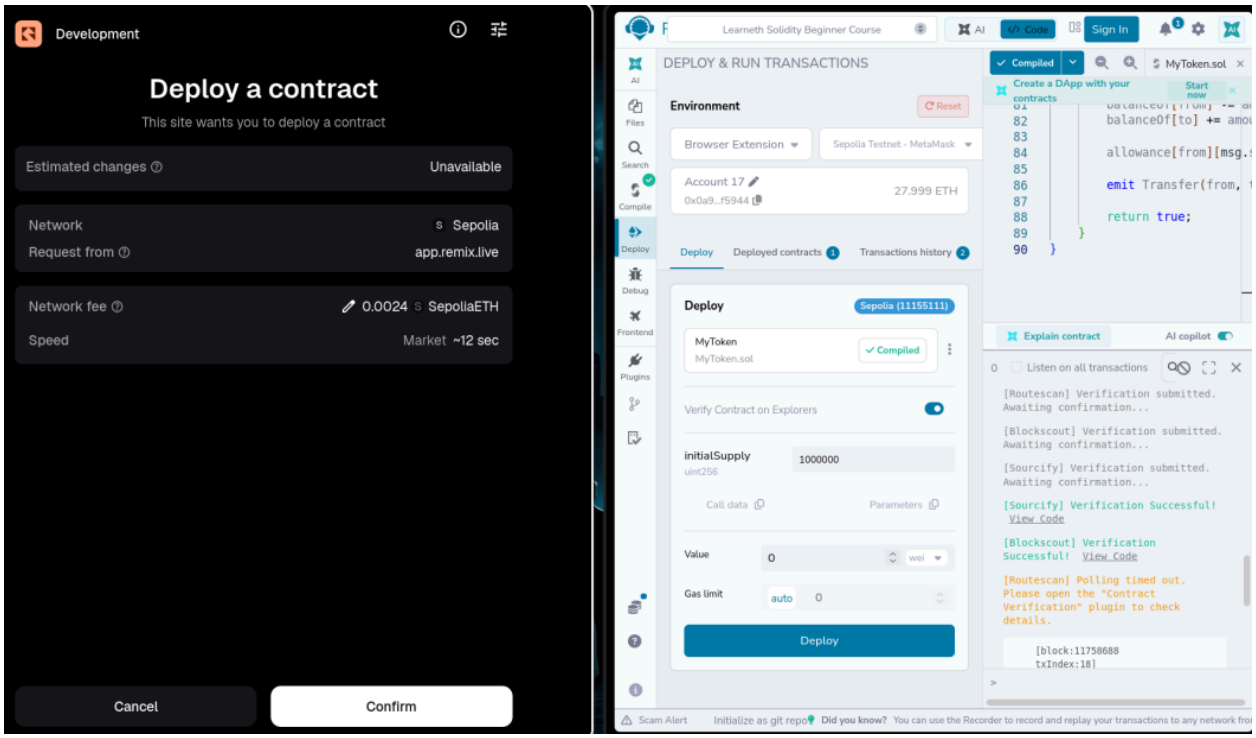


Fig. 3. Deployed MyToken contract with transaction details

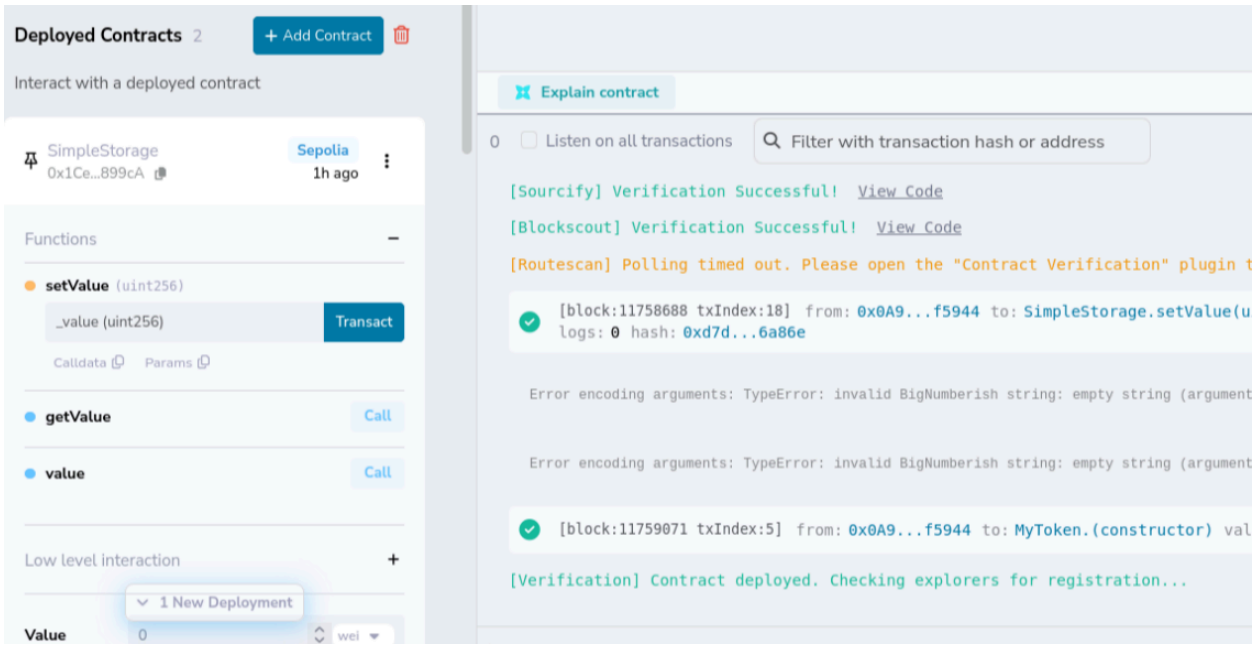


Fig. 4. MyToken compiler panel in Remix IDE

Experiment No. 9

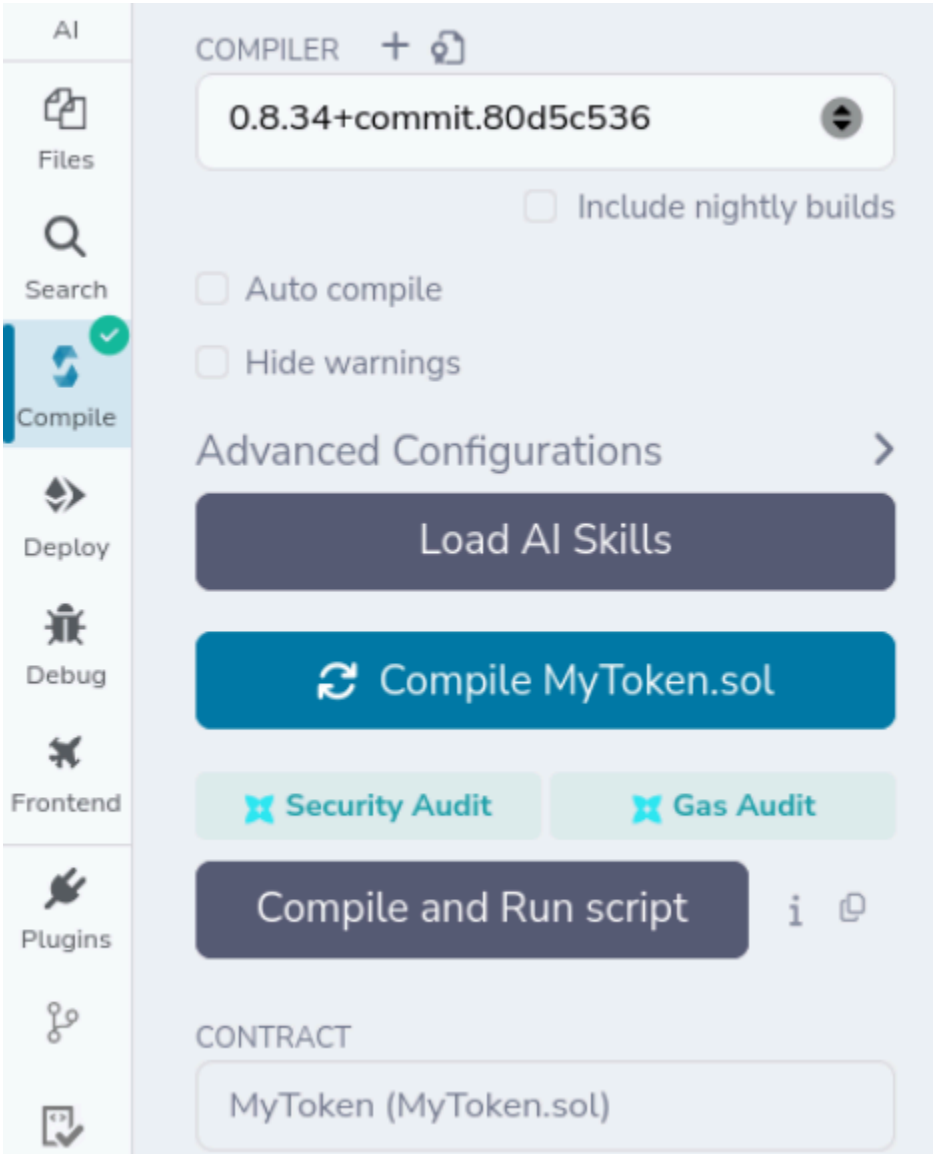


Fig. 5. MyToken public attributes and call results



Fig. 6. MyToken total supply call result

Experiment No. 9

Conclusion:

The `MyToken` smart contract was implemented in Solidity with ERC-20-compatible token properties, balance and allowance mappings, transfer and approval events, and functions for token operations. The screenshots captured the MetaMask deployment approval, contract deployment through Remix IDE, deployed contract details, compilation, and retrieval of public token attributes. This experiment demonstrated the procedure for developing and deploying a fungible token smart contract using Remix IDE connected to MetaMask on a test blockchain network.